

Robust Collaborative LLM Inference in UAV-enabled Wireless Networks

Abstract—

I. INTRODUCTION

Recently, edge artificial intelligence (AI), where intelligence is pushed from the network core to the edge via running AI algorithms over edge devices, is seen as a promising key enabler to meet the great vision of ubiquitous intelligence [1].

FL has demonstrated its empirical success with theoretical convergence guarantees [2]. Many literature have studied how to improve the learning performance [3]–[9].

Our proposed optimization problem yields two main technical challenges. First, edge association has been generally NP-hard. Then combining the resource allocation forms a complex mixed integer nonlinear programming (MINLP) problem with coupled optimization variables. Second, although the objective function could be equal to a set function via some transformation, it is essentially neither submodular nor supermodular, which makes the problem more challenging.

Our main contributions are summarized as follows:

- Firstly, XXX
- Secondly, XXX.
- Thirdly, XXX

II. RELATED WORKS

XXXXXXXXXXXXXXXXXX

III. MODEL AND PROBLEM FORMULATION

In this work, we highlights the deployments of a splitting LLM across UAV swarms. Specifically, we consider a general collaborative edge networks, composed of a total of N UAVs equipped with embedded intelligent computing devices. For the requirements of sensing tasks, these UAVs fly with a constant speed according to the predetermined trajectory, to perform monitoring missions supported by LLMs. Since LLMs usually have a layered architecture consisting of an embedding layer, multiple decoder layer, and an output layer, without loss of generality, we assume the LLM is with L layers. o_l represents the size of activations of layer l , $0 \leq l \leq L$. Let Mem_l be the size of the storage space occupied by a layer $l \in L$ and c_l is the computation requirement. Due to the heterogeneous resource limitation in UAVs, we consider the maximum storage capacity $Mem_n^{\max}$, computation capacity F_n , energy threshold E_n for each UAV $n \in \{1, 2, \dots, N\}$. Based on the model partition technique, the LLM can be divided into multiple submodels and deployed on the UAVs for collaborative inference.

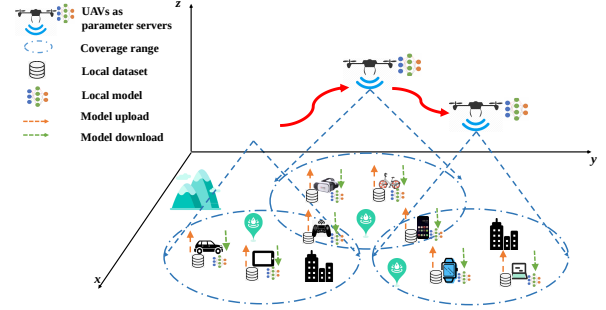


Fig. 1: An illustration of service provisioning in MEC.

A. Latency and Energy Cost Models

To quantify the collaborative inference overhead, we formulate the latency and energy cost models. We introduce the binary decision variable $x_{l,n}$ to denote the LLM deployment strategy. $x_{l,n} = 1$ indicates layer l is deployed on UAV n , otherwise, $x_{l,n} = 0$. There are two kinds of latency in UAV-enabled collaborative task processing, computation latency and communication latency. Let $t_{l,n}^{cmp}$ denote the computation latency of executing inference task for layer l on UAV n . Then we have $t_{l,n}^{cmp} = \frac{c_l}{f_n}$. The corresponding energy consumption is $e_{l,n}^{cmp} = k_0 f_n^2 c_l$, where k_0 represents the energy efficiency parameter. On this basis, for each UAV n , the total computation latency is $t_n^{cmp} = \sum_{l=1}^L t_{l,n}^{cmp} x_{l,n}$, and the total computation

energy consumption is $e_n^{cmp} = \sum_{l=1}^L e_{l,n}^{cmp} x_{l,n}$.

If layer l and layer $l+1$ are deployed on different UAVs, e.g., UAV n and n' , we use $t_{l,n,n'}^{com}$ to denote the communication latency of transmitting the activation parameters of layer l from UAV n to n' . The achievable uplink data rate between UAV n and UAV n' is

$$r_{n,n'} = B_n \log(1 + \frac{p_n h_{n,n'}}{N_0}), \quad (1)$$

where B_n is spectrum bandwidth of each communication channel, p_n is the transmission power of UE n and $h_{n,n'}$ is channel gain between UAV n and n' . Since the data size of activations of layer l is o_l , we have $t_{l,n,n'}^{com} = o_l / r_{n,n'}$, if $n \neq n'$; otherwise $t_{l,n,n'}^{com} = 0$. The corresponding energy consumption is $e_{l,n,n'}^{com} = p_n t_{l,n,n'}^{com} = \frac{p_n o_l}{r_{n,n'}}$, if $n \neq n'$; otherwise $e_{l,n,n'}^{com} = 0$. On this basis, for each UAV n , the total communication latency is $t_n^{com} = \sum_{l=1}^{L-1} \sum_{n'=1}^N t_{l,n,n'}^{com} x_{l,n} x_{l+1,n'}$. The total computation

energy consumption is $e_n^{com} = \sum_{l=1}^{L-1} \sum_{n'=1}^N e_{l,n,n'}^{com} x_{l,n} x_{l+1,n'}$.

The UAVs hover at fixed positions to simultaneously collect data from their coverage areas and perform inference tasks for surveillance objectives. The hovering energy consumption of UAV node n is modeled as

$$e_n^{hov} = P_n^{hov} T^{hov} \quad (2)$$

where P_n^{hov} denotes the hovering power of UAV n and T^{hov} represents the hovering duration. We have

$$T^{hov} = \sum_{n=1}^N (t_n^{cmp} + t_n^{com}). \quad (3)$$

We assume all UAV devices maintain a fixed hovering duration T^{hov} above the monitoring area during task execution. This unified hovering time constraint ensures temporal synchronization across the swarm while satisfying mission continuity requirements.

B. Packet Drop Probability

In this work, we use a cyclic redundancy check (CRC)-based error detection mechanism [?] to identify potential data corruption in the local models received at the edge server. The unreliability of UAV uplink transmission is characterized through the metric of packet drop probability. Concurrently, we adopt the assumption that downlink transmission maintains reliability, ensuring the successful reception of the global model transmitted from the edge server to the UAV. This assumption is justified by the fact that edge servers are typically equipped with significantly higher transmission power capabilities compared to UAVs, thereby guaranteeing robust downlink communication.

Let γ_0 denotes the Signal-to-Noise Ratio (SNR) threshold for successful data decoding. The successful transmission indicator of UAV n is $I_n = \mathbf{I}(SNR_n \geq \gamma_0)$, where $\mathbf{I}(\cdot)$ is indicator function, $SNR_{n,n'} = \frac{p_n h_{n,n'}}{N_0}$. The successful transmission indicator $I_n = 1$ means UAV n successfully transferred the model to the edge server, or 0 otherwise. The packet drop probability (PDP) of UAV n through channel is given by

$$\begin{aligned} PDP_n &= Pr(SNR_n < \gamma_0) = Pr\left(\frac{p_n h_{n,n'}}{N_0} < \gamma_0\right) \\ &= Pr\left(\xi_n < \frac{\gamma_0 N_0}{p_n h_{n,n'}}\right) = 1 - e^{-\frac{\gamma_0 N_0}{p_n h_{n,n'}}} \end{aligned} \quad (4)$$

C. Problem Formulation

The performance of the LLMs is commonly quantified using the perplexity (PPL) metric, a standard means in NLP to evaluate how well a probability model predicts a sample. Given an LLM and a sequence of N tokens, (i.e., $w_1, w_2, \dots, w_M$), the PPL is defined as

$$PPL = \exp\left(-\frac{1}{M} \sum_{k=1}^M \log P_{LLM}(w_1, w_2, \dots, w_{k-1})\right),$$

where $P_{LLM}(w_1, w_2, \dots, w_{k-1})$ denotes the LLM's prediction probability from the previous $k-1$ tokens. A lower PPL signifies superior model performance, demonstrating the models proficiency in accurately predicting the subsequent word in a sequence. Hence, in this context, PPL can serve as a unified metric to assess the impact of channel impairments on the LLMs ability.

Based on the aforementioned models, we define the joint optimization problem of submodel deployment and bandwidth allocation as follows:

$$(\mathbf{P1}) : \underset{\mathbf{X}, \mathbf{B}}{\text{Min}} Q(\mathbf{X}, \mathbf{B}) = \alpha PPL + \beta T^{\text{total}}$$

$$\begin{aligned} &= \alpha \cdot \exp\left(-\frac{1}{K} \sum_{k=1}^K \log P_{LLM}(\omega_k \mid \omega_1, \omega_2, \dots, \omega_{k-1})\right) \\ &+ \beta \cdot \sum_{l=1}^L \sum_{N=1}^N x_{l,n} \cdot t_{l,n}^{cmp} + \sum_{l=1}^{L-1} \sum_{n=1}^N \sum_{n'=1}^N x_{l,n} \cdot x_{l+1,n'} \cdot t_{l,n,n'}^{com} \\ \text{s.t. } &\text{C1 : } \sum_{l=1}^L \text{Mem}_l \cdot x_{l,n} \leq \text{Mem}_n^{\max}, \forall n \in \mathcal{N}, \\ &\text{C2 : } E_n^{cmp} + E_n^{com} + E_n^{hov} \leq E_n^{\max}, \forall n \in \mathcal{N}, \\ &\text{C3 : } \sum_{n=1}^N B_n \leq B^{\max}. \end{aligned}$$

The above optimization problem involves two variables as $\mathbf{X} = \{x_{l,n}\}_{n \in \mathcal{N}, l \in \mathcal{L}}$, $\mathbf{B} = \{B_n\}_{n, n' \in \mathcal{N}}$. Constraint (C1) ensures that the total occupied memory space of deployed layers cannot exceed the maximum storage capacity. Constraint (C2) limits computation energy consumption, transmission energy consumption, and hovering energy consumption during collaborative inference in UAV swarm. Constraint (C3) indicates the allocated bandwidth should not exceed the communication resource capacity.

Complexity Analysis: Unfortunately, it is non-trivial to directly solve the problem P1 due to several challenges. (1) Problem (P1) is a mixed integer programming nonconvex problem, which is hard to solve. It would takes a long time to finding the optimal submodel deployment decision $\mathbf{X}$ due to the exponentially large search space (eg., $N * 2^L$). Besides, the decision variables $\mathbf{X}$ and $\mathbf{B}$ are coupled with each other, which makes it more difficult. (2) When considering the performance-overhead trade-off, the relationship between the inference performance and decision variables is implicit, since there is no specific mathematical analytical formula in the objective function. (3) In practical scenarios, the mobile nature of UAVs often leads to rapidly changing channel state and great variability in noise levels among UAV swarm. This dynamic environment necessitates continuous adaptation of the submodel deployment strategy to maintain inference performance under varying wireless channel conditions.

IV. SOLUTION

A. Problem Transformation

Considering the variability of network conditions, we assume only the wireless channel gains $\mathbf{h} = \{h_{n,n'}\}, n, n' \in \mathcal{N}$

are time-varying in the considered period while other system parameters in Eqs 1-4 are fixed. For each time frame with channel realization $\mathbf{h}$, it falls into the scope of the complex optimization problem in P1, which involves two coupled decision variables. However, once the binary variables $\mathbf{X}$ is given, problem P1 can be reduced to a resource allocation subproblem as follows.

$$\begin{aligned}
(\mathbf{P2}) : \underset{\mathbf{B}}{\text{Min}} \quad & Q(\mathbf{B}) = \alpha PPL + \beta T^{\text{total}} \\
\text{s.t.} \quad & \text{C2} : E_n^{\text{cmp}} + E_n^{\text{com}} + E_n^{\text{hov}} \leq E_n^{\text{max}}, \forall n \in \mathcal{N}, \\
& \text{C3} : \sum_{n=1}^N B_n \leq B^{\text{max}}.
\end{aligned}$$

Naturally, PPL is a constant value with the fixed submodel deployment strategy. Problem P2 is a convex problem only regarding to $\mathbf{B}$, where the optimal bandwidth allocation can be efficiently solved, e.g., using the LP solver in $O(N)$ complexity.

The major difficulty of solving problem P1 lies in the submodel deployment variable $\mathbf{X}$. Traditional optimization algorithms require iteratively adjusting model splitting and deployment decisions towards the optimum, which is fundamentally infeasible for real-time system optimization under dynamically changing channel.

Considering the variability of network conditions and the real-time system optimization in distributed UAV swarm, we reformulate this optimization problem as a sequential decision-making task. To tackle this complex issue, we propose a novel deep reinforcement learning-based online submodel deployment (DROS) algorithm which is adaptive to the evolving environment.

B. The Markov Decision Process

The submodel deployment can be formalized as a Markov Decision Process (MDP), thus we firstly define the three critical factors, namely state space, action space and reward function as follows.

(1) State Space.

The state space $\mathcal{S}$ encompasses current wireless channel gains $\mathbf{h}(t)$, submodel deployment $\mathbf{X}(t)$, namely, $s(t) = \{\mathbf{h}(t), \mathbf{X}(t)\} \in \mathcal{S}$. Through $s(t)$, we can solve problem P2 and obtain the transmission latency, computation latency, as well as the required energy consumption and memory for each inference task at this state.

(2) Action Space.

The action space $\mathcal{A}$ is designed to accommodate a range of possible adjustment of submodel deployment by deciding which layer of the LLM is deployed on which UAV. The submodel deployment is defined as a vector of binary variable $x_{l,n}(t)$, which is presented by: $a(t) = \mathbf{X}(t) = \{x_{l,n}(t)\}_{n \in \mathcal{N}, l \in \mathcal{L}}$. This generalized action space provides the flexibility to optimize the model splitting strategy dynamically in response to varying channel environments.

(3) Reward Function.

We aim to minimize the total inference latency and improve the inference accuracy in the noise environment. Thus, function Q is set as the reward of the our system and we have $R_t = -Q(\mathbf{X}(t), \mathbf{B}(t))$.

C. DROS Algorithm

For dynamically adjusting the splitting of LLMs within UAV networks, we aim to learn an submodel deployment policy $\phi : \mathbf{h} \rightarrow \mathbf{X}^*$ which quickly generates an optimal submodel deployment action $\mathbf{X}^* \in \{0, 1\}^{N \times L}$ of problem P1 once the channel realization $\mathbf{h}$ is revealed at the beginning of each time frame. We employ the DROS algorithm to iteratively learn such policy ϕ from the experience by autonomous training sampling.

Our DROS algorithm includes two alternative stages: deployment action generation and deployment policy update. The generation of the deployment action relies on the use of a DNN, which is parameterized by θ , namely, the weights that connect the hidden neurons. In the t -th time frame, the DNN takes the channel gain $\mathbf{h}(t)$ as the input, and outputs a relaxed deployment action $\hat{\mathbf{X}}$ (each entry is relaxed to continuous between 0 and 1) based on its current deployment policy $\hat{\mathbf{X}}(t)$. The relaxed action is then quantized into K binary deployment actions, among which one best action $\mathbf{X}^*(t)$ is selected based on the achievable reward, related to function Q in P1. The corresponding $\{\mathbf{X}^*(t), \mathbf{B}^*(t)\}$ is output as the solution for $\mathbf{h}(t)$, which guarantees that all constraints listed in (C1)-(C3) are satisfied. The network takes the deployment action $\mathbf{X}^*(t)$, receives a reward $Q(\mathbf{X}^*(t), \mathbf{B}^*(t))$, and adds the newly obtained state-action pair $(\mathbf{h}(t), \mathbf{X}^*(t))$ to the replay memory.

Subsequently, in the policy update stage of the t -th time frame, a batch of training samples are drawn from the memory to train the DNN, which accordingly updates its parameter from $\theta(t)$ to $\theta(t+1)$ (and equivalently the deployment policy $\phi_{\theta(t+1)}$). The new deployment policy $\phi_{\theta(t+1)}$ is used in the next time frame to generate deployment decision $\mathbf{X}^*(t+1)$ according to the new channel $\mathbf{h}(t+1)$ observed. Such iterations repeat thereafter as new channel realizations are observed, and the policy $\phi_{\theta(t)}$ of the DNN is gradually improved. The descriptions of the two stages are detailed as follows.

Deployment Action Generation:

Suppose that we observe the channel gain realization $\mathbf{h}(t)$ in the t -th time frame, where $t = 1, 2, \dots$. The DNN first outputs a relaxed computation offloading action $\hat{\mathbf{X}}(t)$. Thus, we have

$$\hat{\mathbf{X}}(t) = f_{\theta(t)}(\mathbf{h}(t)), \quad (5)$$

where $\hat{x}_{l,n}(t) \in [0, 1]$.

Then, we quantize $\hat{\mathbf{X}}(t)$ to obtain K binary deployment actions, where K is a design parameter. A larger K results in better solution quality and higher computational complexity, and vice versa. The quantization function, g_K , is defined as

$$g_K : \hat{\mathbf{X}}(t) \rightarrow \{\mathbf{X}(k) | \mathbf{X}(k) \in \{0, 1\}^{N \times L}, k = 1, 2, \dots, K\}, \quad (6)$$

Recall that each candidate action $\mathbf{X}(k)$ can achieve $Q^*(\mathbf{h}(t), \mathbf{X}(k))$ reward by solving problem P2. Therefore, the best deployment action $\mathbf{X}(t)^*$ at the t -th time frame is chosen as

$$\mathbf{X}(t)^* = \arg \max_{\mathbf{X}(i) \in \mathbf{X}(k)} Q^*(\mathbf{h}(t), \mathbf{X}(i)). \quad (7)$$

Note that the K -times evaluation of $Q^*(\mathbf{h}(t), \mathbf{X}(k))$ can be processed in parallel to speed up the computation of Eq. 7. Then, the network outputs the deployment action $\mathbf{X}(t)$ along with its corresponding optimal resource allocation $\mathbf{B}(t)$.

Deployment Policy Update:

The offloading solution obtained in Eq. 7 will be used to update the deployment policy of the DNN. Specifically, we maintain an initially empty memory of limited capacity. At the t -th time frame, a new training data sample $(\mathbf{h}(t), \mathbf{X}(t))$ is added to the memory. When the memory is full, the newly generated data sample replaces the oldest one.

We use the experience replay technique [15], [30] to train the DNN using the stored data samples.

Overall, the DNN iteratively learns from the best stateaction pairs $(\mathbf{h}(t), \mathbf{X}(t))$'s and generates better offloading decisions output as the time progresses. Meanwhile, with the finite memory space constraint, the DNN only learns from the most recent data samples generated by the most recent (and more refined) offloading policies. This closed-loop reinforcement learning mechanism constantly improves its offloading policy until convergence. We provide the pseudocode of the DROO algorithm in Algorithm 1.

Algorithm 1: An online DROS algorithm to solve the submodel deployment decision problem.

Input: Wireless channel gain $\mathbf{h}(t)$ at each time frame t , the number of quantized actions K .
Output: Deployment action $\mathbf{X}(t)^*$, and the corresponding optimal resource allocation $\mathbf{B}(t)^*$ for each time frame t .

- 1 Initialize the DNN with random parameters θ_1 and empty memory.
- 2 Set iteration number M and the training interval δ .
- 3 **for** $t = 1, 2, \dots, M$ **do**
- 4 Generate a relaxed offloading action $\hat{\mathbf{X}}(t) = f_{\theta(t)}(\mathbf{h}(t))$.
- 5 Quantize $\hat{\mathbf{X}}(t)$ into K binary actions $\{\mathbf{X}(k)\} = g_K(\hat{\mathbf{X}}(t))$.
- 6 Compute $Q^*(\mathbf{h}(t), \mathbf{X}(k))$ for all $\{\mathbf{X}(k)\}$ by solving (P2).
- 7 Select the best action
 $\mathbf{X}(t)^* = \arg \max_{\mathbf{X}(i) \in \mathbf{X}(k)} Q^*(\mathbf{h}(t), \mathbf{X}(i)).$
- 8 Update the memory by adding $(\mathbf{h}(t), \mathbf{X}(t))$.
- 9 **if** $t \bmod \delta = 0$ **then**
- 10 Uniformly sample a batch of data set
 $\{(\mathbf{h}(\tau), \mathbf{X}(\tau)) | \tau \in \mathcal{T}(t)\}$ from the memory.
- 11 Train the DNN with $\{(\mathbf{h}(\tau), \mathbf{X}(\tau)) | \tau \in \mathcal{T}(t)\}$ and update $\theta(t)$ using the Adam algorithm.
- 12 **end**
- 13 **end**

Adaptive Setting of K :

D. The Reward Surrogate Model for Faster RL

The integration of DROS into our LLM split optimization scenario significantly enhances the efficiency and effectiveness of our RL approach. Nevertheless, the slow reasoning capability of LLM makes the learning process sluggish. Therefore, inspired by the classical model-based reinforce learning, which uses a predictive model to simulate the environment, we adopt a surrogate model to approximate the reward function, thereby boosting the learning efficiency.

V. EVALUATION

A. Evaluation in Energy Saving Performance

Evaluation Setup. We assume all the UEs and UAVs are distributed in the $1000 \text{ m} \times 1000 \text{ m}$ area. If no otherwise stated, the number of UEs is 100. Referring to the setting in [10], the effective computation capacity f_n of each UE is within $[1, 5]$ GHz. The data transmission power is set as $p_n = 1 \text{ W}$. The maximum bandwidth of each edge server is the same as $B_m^{\max} = 2 \text{ MHz}$ [11]. We set the channel gain at the reference distance 1m is $h_0 = -10 \text{ dB}$ and the noise power $N_0 = -90 \text{ dBm}$ [12]. The FL task can be the image classification on a dataset MNIST [?]. The parameters setting of FL task is based on the work in [13]. We suppose the data size of each UE D_n uniformly distributed in $[1000, 1500]$ samples and the number of CPU cycles required per sample is in 3×10^5 cycles/sample. The uploaded model size is assumed to a constant as $s_n = 0.5 \text{ MB}$. For UAVs, the related parameters are set as follows: the fixed altitude $H = 100 \text{ m}$, maximum coverage range $R_{\max} = 100 \text{ m}$. The hovering time is 5 minutes, the working power of each UAV is set as $p_m = 1 \text{ W}$ and the overall energy consumption for edge model aggregation is 100 J . The parameters setting of FL task is based on the work in [13]. We assume that the parameter set as $v = 10$, $\delta = 10$, respectively. Finally, unless stated otherwise, the global accuracy ε are set to be 10^{-1} .

Baseline. We compare the our proposed algorithm XXX with the following three benchmark algorithms.

Random: the client selection and the trajectory decisions are randomly made. UEs are selected randomly, just like the FedAvg algorithm. All of decisions are made under constraints.
? Delay central algorithm: without considering energy consumption constraints, all decisions are made only to minimize the service delay.

? Myopic algorithm: myopic algorithm imposes a stricter constraint to energy consumption during every time slot. The long-term energy constraint can be always satisfied without establishing deficit energy queue. All decisions are made to meet the stricter constraint and minimize service delay.

Based on the the above settings, we run 10 times of different algorithms to average the results.

Evaluation Results. In this section, we demonstrate the convergence behavior of the alternating optimization-based algorithm and study the influence of number of UEs, the coverage range, the communication bandwidth, the local data size and the maximum training latency constraint on the overall energy consumption.

1) Convergence: Fig. ?? demonstrates the convergence behavior of Algorithm 4, which is devised to obtain the energy consumption within an edge server. We can observe that compared with the initial scheme, the energy consumption is optimized by 35.01%, 36.48%, and 33.97% under different data volumes as similar as possible. Our proposed iteration optimization algorithm can greatly reduce the E_m^{edge} and converge fast within finite iterations.

VI. CONCLUSION

REFERENCES

- [1] W. Saad, M. Bennis, and M. Chen, "A vision of 6g wireless systems: Applications, trends, technologies, and open research problems," *IEEE Network*, vol. 34, no. 3, pp. 134–142, 2020.
- [2] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," *Proc. MLSys*, 2020.
- [3] L. Liu, J. Zhang, S. Song, and K. B. Letaief, "Client-edge-cloud hierarchical federated learning," in *Proc. IEEE ICC*, 2020, pp. 1–6.
- [4] Z. Wang, H. Xu, J. Liu, H. Huang, C. Qiao, and Y. Zhao, "Resource-efficient federated learning with hierarchical aggregation in edge computing," in *Proc. IEEE INFOCOM*, 2021, pp. 1–10.
- [5] X. Ouyang, Z. Xie, J. Zhou, J. Huang, and G. Xing, "Clusterfl: a similarity-aware federated learning system for human activity recognition," in *Proceedings of the 19th AICMAS*, 2021, pp. 54–66.
- [6] C. T. Dinh, N. H. Tran, M. N. Nguyen, C. S. Hong, W. Bao, A. Y. Zomaya, and V. Gramoli, "Federated learning over wireless networks: Convergence analysis and resource allocation," *IEEE/ACM Transactions on Networking*, vol. PP, no. 99, pp. 1–12, 2020.
- [7] L. Yang, Y. Lu, J. Cao, J. Huang, and M. Zhang, "E-tree learning: A novel decentralized model learning framework for edge ai," *IEEE Internet of Things Journal*, vol. 8, no. 14, pp. 11 290–11 304, 2021.
- [8] L. Yang, Y. Gan, J. Cao, and Z. Wang, "Optimizing aggregation frequency for hierarchical model training in heterogeneous edge computing," *IEEE Transactions on Mobile Computing*, 2022.
- [9] X. Zhou, W. Liang, J. She, Z. Yan, I. Kevin, and K. Wang, "Two-layer federated learning with heterogeneous model aggregation for 6g supported internet of vehicles," *IEEE Transactions on Vehicular Technology*, vol. 70, no. 6, pp. 5308–5317, 2021.
- [10] S. Luo, X. Chen, Q. Wu, Z. Zhou, and S. Yu, "Hfel: Joint edge association and resource allocation for cost-efficient hierarchical federated edge learning," *IEEE Transactions on Wireless Communications*, vol. 19, no. 10, pp. 6535–6548, 2020.
- [11] Y. Zhou, C. Pan, P. L. Yeoh, K. Wang, M. Elkashlan, B. Vucetic, and Y. Li, "Secure communications for uav-enabled mobile edge computing systems," *IEEE Transactions on Communications*, vol. 68, no. 1, pp. 376–388, 2019.
- [12] F. Zhou, Y. Wu, H. Sun, and Z. Chu, "Uav-enabled mobile edge computing: Offloading optimization and trajectory design," in *IEEE ICC*, 2018, pp. 1–6.
- [13] Z. Yang, M. Chen, W. Saad, C. S. Hong, and M. Shikh-Bahaei, "Energy efficient federated learning over wireless communication networks," *IEEE Transactions on Wireless Communications*, vol. 20, no. 3, pp. 1935–1949, 2020.